hw02

Please make sure you're following all the instructions described in the "Homework Instructions" page on Canvas (go to the homepage of our class and you'll see a link saying "Homework Instructions" just underneath the header of the page).

1 Vector basics

1.1 *Number of names*

Create a vector containing the names Frodo, Sam, Legolas, Gimli, Boromir, Pippin, Merry, Aragron, and Gandalf (in that order). Then use an appropriate function to count the number of names in the vector.

vars:

- `fellowship` - vector containing names
- `n_fellowship` - count of the number of names

1.2 *Indexing*

Use vector indexing to create a new vector containing the names Frodo, Sam, Pippin, and Merry (in that order).

var: `hobbits`

1.3 *Removing elements*

Use vector indexing (using negative indices) to create a new vector (from `fellowship`) with the names Gandalf the Grey and Boromir removed.

var: `fellowship2`

1.4 *Adding elements*

Use the `c()` function to create a new vector that's the result of appending `fellowship2` with the string "Gandalf the White".

var: `fellowship3`

2 Repetition

2.1 *Lots of fives*

Create a vector that contains the number 5 repeated 100 times (note: you shouldn't type out 100 5's).

var: `fives`

2.2 *Repeating two numbers*

Create a vector that contains 100 1's followed by 100 5's. Use a single call to `rep()`. You may want to check the documentation (`?rep`) if you're not sure how to do this. I'd recommend scrolling to the bottom to look at the examples. Beneath the "Examples" header is a "Run examples" link - if you click on this, you'll see the output from the example code. This will help you identify which options you should be using.

vars: `onefive`

3 Addition

Create a vector containing the numbers 1 through 30 (how can we do this without typing out all 30 numbers?). Next, add 1 to every third number (elements, 3, 6, 9, etc. should each be incremented by 1). Be sure to make use of recycling.

var: `nums` - result of adding 1 to every third number of 1 through 30

4 Sequences

For this question, you'll use the `seq()` function to generate sequences of numbers. You can provide arguments to this function in a couple different ways. For example, the following are all valid usages of `seq()`:

```
seq(from = 1, to = 7, by = 3)
seq(from = 1, to = 7, length.out = 10)
seq(from = 1, by = 2, length.out = 50)
```

4.1 *Sequence 1*

Create a sequence of 23 evenly-spaced numbers where the minimum value is 1 and the maximum value is 97.

var: `seq1`

4.2 *Sequence 2*

Create a sequence of multiples of 7, starting with 7 and ending with 77.

var: `seq2`

4.3 *Sequence 3*

Create a sequence containing the first 25 multiples of 3 (it should start with 3).

var: `seq3`

5 Square root and `NaN`

5.1 *Square root*

First create a vector of all even numbers from -8 to 24 (inclusive - first number should be -8 and last number should be 24).

Then use `sqrt` to find the square root of these numbers.

var: `sqrt_even`

5.2 *Mean*

Take the mean of the vector. You'll need to remove the `NaN`s in order to calculate the mean - check the documentation of `mean()` (i.e. run `?mean`) to determine how to do this.

6 Summary statistics

Use the appropriate R functions to find the mean, standard deviation, and variance of the numbers (3,1,5,7,4,2,4,7,9).

vars:

- `mn`: the mean
- `st`: the standard deviation
- `vr`: the variance

7 Estimating π

(Gagolewski, Exercise 2.12)

The expression:

$$4\sum_{i=1}^{n}\frac{(-1)^{i+1}}{2i-1} = 4\left(\frac{1}{1}-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+\cdots\right)$$

slowly converges to π as n approaches ∞. Calculate it for $n = 1,000$ using the vectorized functions we've discussed, making use of the recycling rule as much as possible.

vars: `pi_est`

8 Generating data

8.1 *Create `x`*

R provides a number of functions for working with probability distributions. In particular, it provides functions for drawing random numbers from a distribution. (You can run `?distributions` to see a list of the distributions). For this question we'll use the function `rnorm()` to draw values from a normal distribution ("r" stands for random and "norm" is short for "normal") with mean 10 and standard deviation 3 (check `?rnorm` for information about the arguments). After generating the numbers, use `hist(x)` to create a histogram of the values.

vars:

- `x`: 100 random numbers

Before generating the numbers, set the seed of the random number generate to 212 (`set.seed(212)`). This ensures that your randomly generated numbers will be the same as those used in the answer key. Make sure this line of code appears before you generate the random numbers using `rnorm()`.

8.2 *Calculate `y`*

Suppose that the values in `x` (from the previous question) are x coordinates for a line with slope 2 and intercept 3. Calculate the y coordinates. Afterwards, plot the resulting line using the `plot()` function (the first argument should be the x coordinates and the second argument should be the y coordinates).

var: `y`

8.3 *Add error*

Let's add some random error to the y coordinates. Using the `rnorm()` function, add random error to each of the y coordinates. Use `mean = 0`, and `sd = 6`. Like in the previous question, create a scatter plot, this time using the modified y coordinates.

var: `y2` - y coordinates with error added.

Use `set.seed(1)` before generating the random numbers.

8.4 *MSE*

Suppose that `y` represents the predicted values and `y2` represents the actual values. The mean square error (MSE) for a set of predictions is defined as:

$$\frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n}$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and n is the number of observations. Calculate the MSE.

vars: `mse`

8.5 *Covariance*

The sample covariance between two variables x and y (both of length n) can be calculated as

$$\frac{\sum_{i=0}^{n}(x_i - \bar{x})(y_i - \bar{y})}{n-1}$$

Write code to calculate the covariance between `x` and `y2` from the previous questions. Then use the function `cov()` to confirm that your calculation is correct.

var: `cov_x_y2` - the covariance between `x` and `y2`, calculated *without* `cov()`

9 Rescaling values

9.1 *Mean 0, SD 1*

We're often interested in rescaling a vector of numbers. In some cases we're interested in rescaling the values so that the new mean is 0 and the new standard deviation is 1, which we can do as follows:

$$\frac{x - \bar{x}}{s}$$

where x is a vector, $\bar{x}$ is the mean of x, and s is the standard deviation of x.

Create a vector that contains the following numbers:

$2, 4, 5, 1, 2, 3, 4, 1, 2$

Scale this vector using the formula above.

vars: `scaled1`

9.2 *Rescale to [0,1]*

We're also sometimes interested in rescaling a vector so the minimum value is 0 and the maximum value is 1. Determine how to rescale the values this way and then write code to do this for the same vector you used in the previous question.

vars: `scaled2`